

Algorithm Benchmarking and Outcomes

**Society Connectedness & Health Lab
Yale School of Public Health**

Church Closures Dashboard Project



Table of contents

Generating the Dashboard Map Metrics from the 2026 Format Data	2
Overview	2
Important Considerations	3
Ignored Files	3
Environment Reproducibility	3
Methods	6
Construct The Denominator Reference Tables	6
Square Miles	8
Population Counts	8
Prepare the Analytic Dataset	10
Metric Generation	10
Total and Missing Values	11
Closures and Reopenings	12
Relocations	12
Rollups	12
Results	14
Appendix	15
GEOID Square Mile Lookup Table	15
ZCTA Square Mile Lookup Table	15
GEOID Total Population Lookup Table	16
ZCTA Total Population Lookup Table	16
Analysis Ready Dataset	17
Final Results	17
Confirm Deliverables Subset	20
References	20

Generating the Dashboard Map Metrics from the 2026 Format Data

Prepared By: [Shelby Golden, M.S.](#) from the Yale School of Public Health [Data Science and Data Equity](#) team

Date Created: August 30th, 2026

Date Updated: September 1st, 2026

Overview

The project objectives were as follows:

1. **REVIEW:** Evaluate the raw data and previously outlined cleaning, validation, and harmonization methods employed by [Dr. Insang Song](#) for accuracy and identify opportunities for improvement.
2. **PREPARATION:** Implement identified improvements to prepare the dataset for analysis and distribution to qualifying collaborators.
3. **METRICS:** Calculate the metrics necessary for dynamic visualization.

Additionally, the project Principal Investigator (PI), [Professor Yusuf Ransome](#), requested the pipeline be reproducible for future collaborators and readily adaptable to potential changes in project scope.

Following receipt of the raw data in Summer 2025, a review was conducted and findings were compared against the original methodology provided by Dr. Song. When an updated dataset was subsequently delivered in May 2026, the pipeline was expanded to accommodate two designated variations: the **2023 Format** and the **2026 Format**.

This document provides an overview of the methodological changes implemented following the initial prototype developed using the 2023 Format, and to accommodate the differences identified in the 2026 Format Review. While many of the fundamental methods were retained, notable changes include implementing identified points of improvement from the prototype, the exclusion of persistence, and the inclusion of additional metrics such as relocations.

Results generated from this pipeline were used for the Fall 2026 [dashboard](#) developed by

Gordon Tu with demographics by Nick Begotka.

Important Considerations

Selected code excerpts are referenced below. The complete code used to generate this report can be found in “[Code/Generate the Metrics_2026 Format.R](#)” with all packages required documented in the **SET UP THE ENVIRONMENT** section. Custom functions used throughout the script are in “[Code/Support Functions/For Generate the Metrics_2026 Format.R](#)”.

Ignored Files

Individual-level files are withheld from GitHub and Git tracking in accordance with the Data Use Agreement (DUA) with the data provider. Many of these are stored in `**/KEEP LOCAL/` directories or are explicitly listed under the “Project-Specific Exclusions” section of the `.gitignore` file.

Most files are available on the SOCAH Lab OneDrive for the project, while others are user-specific and do not need to be shared. If there are local-only files for this project, they will be added to the “Church-Closures-Dashboard GitHub LOCAL ONLY Files” directory. This directory will contain `README.md` files specifying the intended use and filepath for each file.

If you do not have access to the SOCAH Lab OneDrive, carefully follow the codebase instructions to establish the repository and replicate results on your own device. Raw data access instructions may vary depending on your institution.

Environment Reproducibility

Results were produced using R (v4.5.2), RStudio (v2026.01.1+403), and Quarto (v1.8.27). The `renv` package (v1.1.4) is included to reproduce the coding environment, capturing all relevant packages and their versions. If issues arise when running the scripts, verify that the environment is initialized and that the correct versions of R, RStudio, and Quarto are being used.

⚠ Compatibility

Updating packages, R, or Quarto may introduce compatibility issues with legacy code that will need to be resolved on a case-by-case basis.

First-time users should initialize the environment to install all required packages and ensure a reproducible coding environment. To do so:

1. In the command-line application (i.e., Terminal for Macs or Git Bash for Windows) navigate to the codebase file location.

Command-Line Interface

```
cd "FILE-PATH/Church-Closures-Dashboard/"
```

2. Launch the project by opening the *.Rproj file in RStudio.
3. In the R console, activate the environment by running the following lines of code:

Command-Line Interface

```
renv::restore() # Download packages and their version saved in the
↪ lockfile.
```

i Note

If you are asked to update packages, say no. `renv` is intended to recreate the same environment under which the project was created, making it reproducible. You are ready to proceed when running `renv::restore()` gives the output:

RStudio Console

```
- The library is already synchronized with the lockfile.
```

If you experience any trouble with this step, you might want to confirm that you are using R (v4.5.2) in the RStudio (v2026.01.1+403) and Quarto (v1.8.27). You can also read more about `renv` in their [vignette](#)¹.

Methods

Four metrics were requested for dashboard visualization:

1. Number of religious businesses closed
2. Rate of religious businesses closed per 10,000 persons
3. Number of religious businesses closed per square mile

The following additional metrics were requested but were not intended for map visualization:

1. Number of religious businesses open
2. Number of religious business reopenings following a closure
3. Number of religious businesses that relocated, i.e., those filed under a different address
4. Number of religious businesses removed due to missing information: missing `address_line_1`, a PO Box listed as `address_line_1`, or missing geocoordinates

Persistence was excluded as a metric following user review, where it was flagged as a confusing outcome. All metrics were calculated simultaneously. Counts were first prepared over all qualifying businesses open during a given date range, defined as those with at least one non-zero entry, at the block group level, the finest census boundary resolution available. These counts were then aggregated to coarser resolutions, and rates were calculated following this aggregation. No aggregation was required prior to rate calculation for the block group and ZIP Code Tabulation Area (ZCTA) census boundaries.

Metrics are calculated across all user-selectable date ranges from 2000 to 2025; possible combinations are generated by `build_year_windows()`. The minimum selectable range is five years, as closure event detection requires identifying businesses that have not filed for four or more consecutive years. Applying this constraint to all valid start and end year combinations within the 2000–2025 period yields 253 possible date ranges.

Construct The Denominator Reference Tables

Two metrics require denominators that vary across census boundaries and decennial years: rates per 10,000 persons and number of closures per square mile. To improve the efficiency of downstream calculations, these denominators were pre-compiled into a lookup table.

Additionally, processing each census boundary level independently is unnecessary, as population and square mileage can be summed for higher-level aggregated GEOIDs. The only

exception is ZCTAs, which must be calculated independently since their polygon boundaries do not cleanly align with GEOIDs. Under this approach, the smallest GEOID, block, was used to maximize future metric generation.

To efficiently handle block-level GEOID data, the state-level [TIGER/Line Shapefiles](#) manually downloaded during **Step 2** of the data cleaning and validation process were imported by census division (1–9), each grouping as many as nine states². These were then compiled into a single lookup table containing both metrics.

i Census Boundary Files: Download and Local Build Requirements

All census boundary files, both the downloaded raw shapefiles and the compiled GeoPackage versions, are too large for effective Git tracking and distribution. New users will need to download the source files independently and generate the GeoPackage files locally. For download instructions and links to sources, refer to the [Directory Specific Notes](#) in the project GitHub repository. For the coded implementation, refer to **SUBSECTION A3: Build Precompiled TIGER/Line GeoPackages** in [“Clean Raw Data_Step 2_2026 Format.R”](#).

Because this process is involved, a dedicated set of functions was created for the generation of these reference shapefiles: [“Support Functions/For Step 2_Compile Decennial Data.R”](#). The custom function `preflight_tiger_outputs()` first assesses the local environment for the status of shapefile downloads and builds, flagging any missing files or incomplete downloads to the user. See **SUBSECTION A4: Build Precompiled TIGER/Line GeoPackages** in [Clean Raw Data_Step 2_2026 Format.R](#) for the implementation.

In brief, standardized decennial spatial layers for all census boundaries are compiled using `standardize_geo_layer()` for block and block group-level state GEOIDs, and `standardize_core_area_layer()` for Metropolitan/Micropolitan Statistical Areas (denoted as CBSA), Combined Statistical Areas (CSA), and ZCTAs. These standardized files are then written using `write_state_gpkg()` for GEOIDs and `write_core_areas_gpkg()` for CBSAs, CSAs, and ZCTAs, following the execution of their respective `build_cbsa_csa()` and `build_zcta()` functions.

For size considerations, the GEOID files are retained separately by state, while the CBSA, CSA, and ZCTA layers are compiled as nationally representative files.

Square Miles

Square mileage can be calculated from polygon boundaries in TIGER/Line Shapefiles using `set_units(st_area(x_ea), "acre")`, then converted to square miles^{3,4}. Because census GEOIDs are non-overlapping, this calculation is only required for the smallest available boundary, block. These are then summed to aggregate into larger GEOID census boundaries. ZCTAs are processed separately by the same method and are not aggregated into other census boundaries, as their polygon boundaries do not cleanly cross-walk with standard GEOIDs.

When compiling lookup tables by GEOID, each TIGER/Line shapefile for a state block created for **Step 2** in a given division is imported using the custom function `read_state_gpkgs_for_data()`. A progress bar tracks processing using the custom function `count_sf_in_blocks_by_state()` to count spatial objects per division. Lookup tables are stratified by GEOID and decennial year. Since each state-level shapefile contains lists for each decennial year, `get_year_from_name()` extracts the available years (2000, 2010, 2020) and `area_table_one(x, yr, id_col = "geoid")` computes a per-object area result table for each.

ZCTA lookup tables are constructed similarly using the core areas slim file from **Step 2**, which contains CBSA, CSA, and ZCTA data. Only ZCTA lists are processed: `get_year_from_name()` extracts the available decennial years and `area_table_one(x, yr, id_col = "area_code")` computes the per-object area result table.

In both cases, per-state/per-year result tables are appended into by-year tables as each division is processed. Once all divisions are complete, results are stacked within each year into a single deduplicated table using `purrr::imap()` and `bind_rows()`. Year-specific tables are then merged side-by-side on GEOID or ZCTA to produce a final sorted wide master table⁵.

Refer to Appendix - [GEOID Square Mile Lookup Table](#) and [ZCTA Square Mile Lookup Table](#) for information on where to find the code that generates these results, as well as the results codebook.

Population Counts

Total population values are pulled at the block level using `tidycensus`⁶. As shown in the prototype methods, the 2020 decennial year draws total population from a different table than 2000 and 2010. While 2000 and 2010 use the same variable name, "P001001", each refers to its own vintage-specific table. The following tables were used to pull total population for each

respective decennial year:

RStudio Console

```
# SUBSECTION A2: Get Population Counts

# Define the variable IDs and reference tables used by tidycensus to pull
# total population for each decennial year.
spec <- tribble(
  ~year, ~sumfile, ~var_totpop,
  2000, "sf1", "P001001", # 2000 Summary File 1 ("sf1")
  2010, "sf1", "P001001", # 2010 Summary File 1 ("sf1")
  2020, "dhc", "P1_001N" # 2020 Demographic and Housing Characteristics
    ↪ File ("dhc"), Table P1
)
```

Values for block-level GEOIDs are built as a three-level ladder that breaks up the `tidycensus` API pull into smaller buckets, finally getting compiled into one master file similar to the square mile lookup table⁶. First, the custom function `get_block_pop_year_by_county()` requests one county in one state for one decennial year to return a minimal lookup: GEOID plus a `popYYYY` column for each decennial year. `get_block_pop_state_year()` then uses that building block by first listing all counties in the state (via `tigris::counties()`), and calling `get_block_pop_year_by_county()` for each county, row-binding the results into a complete state-wide block table for that year⁷.

Finally, `get_block_pop_all_states()` orchestrates the full run: it loops over states and over the year/spec rows, calls `get_block_pop_state_year()` to produce one table per year, joins those year tables side-by-side on GEOID to make a wide per-state dataset, and then stacks all states into one deduplicated, sorted block-level population master.

ZCTA population values follows a simpler two-step workflow. The custom function `get_zcta_pop_year()` takes a decennial year and fetches ZCTA-level total population using `tidycensus::get_decennial(geography = "zcta", geometry = FALSE, output = "wide")`, then standardizes the identifier from GEOID to ZCTA and renames the estimate to a year-labeled `popYYYY` column. `get_zcta_pop_all_decennials()` then iterates over each row of the spec table using `purrr::pmap()`, collecting the resulting decennial year-specific values, merging them side-by-side into one master table of ZCTA and `popYYYY` columns, sorted by ZCTA⁵.

Refer to Appendix - [GEOID Total Population Lookup Table](#) and [ZCTA Total Population Lookup Table](#) for information on where to find the code that generates these results, as well as the results codebook.

Prepare the Analytic Dataset

In the final data cleaning and validation step (**Step 3**), the cleaned and validated data were reshaped and annotated with metrics that support metric generation. Select metadata and quality control outputs from earlier steps were retained through that process but are not needed for generating the metrics that follow, so they are excluded here.

The geography columns for GEOID and ZCTA were pivoted from wide format (`geoid_[2000|2010|2020]` and `zcta_[2000|2010|2020]`) to long format using `melt()`, so that each row represents one ABI/address observation for a single decennial year, with unified `geoid` and `zcta` columns alongside `year`⁵. This structure allows the reference denominators — area in square miles and total population — to be joined by decennial year when computing the per-square-mile and per-10,000-persons metrics in subsequent steps. Finally, the year variable was recoded to an integer, columns were reordered so that `abi`, `address`, `year`, `geoid`, and `zcta` appear first, and rows were sorted by `abi`, `address`, and `year`.

Both GEOID lookup tables described in the previous two sections were intentionally compiled at the smallest available census boundary level, blocks. These are pre-aggregated to the block-group level present in the dataset prior to joining. The aggregation algorithm in **SUBSECTION B2: Aggregate Block-Level GEOIDs to Dataset Boundary Level** does not assume a specific geography level, as preceding steps support processing at either the block or block-group level. It is assumed, however, that only one boundary level will be present at a time. While blocks or block groups are the expected units, this step agnostically inspects GEOID length and aggregates accordingly. ZCTA does not require pre-aggregation.

Refer to [Appendix - Analysis Ready Dataset](#) for information on where to find the code that generates these results, as well as the results

Metric Generation

Metrics were calculated across different date range selections, religious affiliations, and decennial periods. At each step, the dataset was subset to only the data relevant to that stratification: rows in which all values were zero were removed after selecting columns for

the given date range and then for ABI with any religious affiliation matching that run. Filtering was performed using the custom function `filter_ts()` to remove rows containing all zeros, and `abi_any_true()` to remove ABI records with no entries matching the religion being calculated.

RStudio Console

```
# Inside "SUBSECTION C2: Generate the Metrics in Chunks" for loop

# Available religion classifiers.
relig_cols <- c(
  "buddhist_temple", "christian_church", "hindu_mandir", "jewish_synagogue",
  "muslim_mosque", "sikh_gurdwara", "other_religion", "interfaith",
  ↪ "unspecified"
)
```

Total and Missing Values

Following filtering, all remaining ABIs are counted and checked for critical missing values. Any ABI containing critical missing values is excluded from subsequent metric calculations.

RStudio Console

```
# Inside "SUBSECTION C2: Generate the Metrics in Chunks" for loop
# ~ line 837

# (1b) flags by (geoid, zcta)
flag_counts <- church_relig[
  ,
  .(
    n_open      = uniqueN(abi),
    addr1_na    = uniqueN(abi[addr1_is_na    %in% TRUE]),
    addr1_pobox = uniqueN(abi[addr1_is_pobox %in% TRUE]),
    geoid_na    = uniqueN(abi[geoid_is_na    %in% TRUE]),
    zcta_na     = uniqueN(abi[zcta_is_na     %in% TRUE])
  ),
]
```

```
by = .(geoid, zcta)
]
```

Closures and Reopenings

A closure is defined as an event in which four or more consecutive years have no filings under a given address. Closures were calculated both including and excluding ABI that filed under different addresses within that date range. When ABI with a relocation were included, the metric was aggregated column-wise into a single row-vector from which the number of closures was calculated.

A reopening is defined as at least two consecutive years of activity following a closure. Reopenings are calculated both including and excluding ABI that relocated within the date range. Closures and reopenings were calculated using the custom function `calculate_closure(min_zero_run = 4L, min_one_run_reopen = 2L, multi_addr_mode = c("skip", "compress"))`.

Relocations

A relocation is defined as any instance in which a religious business files under more than one address within the date range. Relocation metrics include: the number of ABI that relocated at least once; the average number of relocations per ABI, defined as the number of distinct addresses minus one; the move-count-weighted mean distance across all relocations; the maximum single-move distance recorded for any ABI; and binary flags indicating whether any move for a given ABI exceeded 5, 10, or 25 miles. Relocations were calculated using the custom function `calculate_closure(multi_addr_mode = c("compress"))`.

Rollups

After all metrics are calculated for each religion, results are aggregated using the custom function `rollup_results()`. This process rolls up metrics calculated at the smallest geographic boundaries to progressively coarser census boundaries. Block group GEOID calculations are aggregated to the tract, county, and state levels; however, ZCTA results

are not rolled up. Aggregations are performed across each unique combination of GEOID, decennial year, and religion.

Rather than summing raw closure and reopening events counts across ABI, each is summarised into three derived outputs: any — the count of ABI with at least one event; avg — the mean number of events per ABI among those with at least one event; and max — the maximum number of events recorded for any single ABI.

Non-event columns follow custom aggregation rules: move threshold flags (i.e. moves greater than 5, 10, or 25 miles) are summed as counts of flagged ABI, total moves are averaged, weighted mean distances are averaged, and maximum move distances are taken as the maximum across all ABI. Finally, closure rates are derived from the any columns and appended as both per-10,000-population and per-square-mile rates, using the appropriate denominators for each geographic level.

Following all aggregations, the resulting columns are reordered using the custom function `reorder_rollup_cols()` for clarity according to the following structure:

1. Geographic identifiers, year, religion, and denominators appear first.
2. Window-specific event columns follow the following order, where `<lab>` represent the date range selection:
 - `n_open__<lab>` — count of open ABI
 - No-move closures: any, rates, avg, max
 - No-move reopenings: any, avg, max
 - `n_move__<lab>` — count of ABI with moves (inserted after `reopenings_no_move_max__<lab>`)
 - All closures: any, rates, avg, max
 - All reopenings: any, avg, max
3. All remaining columns appear last, including move totals, distances, and threshold flags.

Refer to [Appendix - Final Results](#) for information on where to find the code that generates these results, as well as the results codebook.

Results

SUBSECTION C3: Validate Chunk Coverage assesses whether all expected metrics were calculated across all chunks using three pre-flight checks. Pre-flights A, B, and C confirm that the chunks written by the preceding metric generation function cover all expected combinations of metrics, metric $\times$ date range, and geographic identifiers.

- Pre-flight A confirms that the date labels embedded in each folder's column names match `date_range$label` exactly.
- Pre-flight B confirms that every metric $\times$ date column pair expected from `metric_order  $\times$  date_order` is present in each folder's schema, with no duplicates.
- Pre-flight C confirms that every GEOID and ZCTA value is exactly the Census-standard character length for its geography type.

One limitation of this approach is the size of the resulting output files. Handling all metrics calculated across every geographic boundary simultaneously is preclusive, particularly at finer geographic resolutions. As a result, the files must be used strategically, rather than loading all data into memory at once. **SUBSECTION D3: Compile Results for Three Date Ranges** demonstrates one approach to managing this over multiple chunks.

Appendix

GEOID Square Mile Lookup Table

For the implementation, refer to **SUBSECTION A1: Get Square Miles** in “[Code/Generate the Metrics_2026 Format.R](#)”. All 50 states and the District of Columbia block-level GeoPackage files contain a spatial geometry column with boundary polygons for each census unit, represented as an `sf` geometry object, were converted to square mile area lookup tables. Results were written to `./Data/Results/KEEP LOCAL/From Generate the Metrics/GEOID Square Miles_Block Level_08.18.2026.parquet`.

Table 1: Codebook for TIGER/Line-Based Square Mileage Estimates

Field	Description
<code>geoid</code>	Block-level GEOID covering the entire US and District of Columbia.
<code>area[2000 2010 2020]_mi2</code>	Calculated square mileage for each decennial period's GEOID mapping.

ZCTA Square Mile Lookup Table

For the implementation, refer to **SUBSECTION A1: Get Square Miles** in “[Code/Generate the Metrics_2026 Format.R](#)”. All 50 states and the District of Columbia ZCTA GeoPackage files contain a spatial geometry column with boundary polygons for each census unit, represented as an `sf` geometry object, were converted to square mile area lookup tables. Results were written to `./Data/Results/KEEP LOCAL/From Generate the Metrics/ZCTA Square Miles_08.18.2026.parquet`.

Table 2: Codebook for TIGER/Line-Based Square Mileage Estimates

Field	Description
<code>zcta</code>	ZIP Code Tabulation Areas (ZCTA) covering the entire US and District of Columbia.
<code>area[2000 2010 2020]_mi2</code>	Calculated square mileage for each decennial period's ZCTA mapping.

GEOID Total Population Lookup Table

For the implementation, refer to **SUBSECTION A2: Get Population Counts** in “Code/Generate the Metrics_2026 Format.R”. Total population counts at the block level were retrieved via `tidycensus`, which queries the US Census Bureau using a specified table and variable. The 2000 and 2010 decennial census data were drawn from Summary File 1 (`sf1`), Table P001, variable P001001. The 2020 decennial census data were drawn from the Demographic and Housing Characteristics File (`dhc`), Table P1. Results were written to “./Data/Results/KEEP LOCAL/From Generate the Metrics/GEOID Population_Block Level_08.20.2026.parquet”.

Table 3: Codebook for Block-Level Decennial Population (`tidycensus`)

Field	Description
<code>geoid</code>	Block-level GEOID covering the entire US and District of Columbia.
<code>pop[2000 2010 2020]</code>	Total population count for each decennial census year at the block level.

ZCTA Total Population Lookup Table

For the implementation, refer to **SUBSECTION A2: Get Population Counts** in “Code/Generate the Metrics_2026 Format.R”. Total population counts for ZCTA census boundaries were retrieved via `tidycensus`, which queries the US Census Bureau using a specified table and variable. The 2000 and 2010 decennial census data were drawn from Summary File 1 (`sf1`), Table P001, variable P001001. The 2020 decennial census data were drawn from the Demographic and Housing Characteristics File (`dhc`), Table P1. Results were written to “./Data/Results/KEEP LOCAL/From Generate the Metrics/ZCTA Population_08.20.2026.parquet”.

Table 4: Codebook for ZCTA-Level Decennial Population (`tidycensus`)

Field	Description
<code>zcta</code>	ZIP Code Tabulation Areas (ZCTA) covering the entire US and District of Columbia.
<code>pop[2000 2010 2020]</code>	Total population count for each decennial period’s ZCTA mapping.

Analysis Ready Dataset

For the implementation, refer to **PART B: FINAL RESHAPE AND JOIN REFERENCE DENOMINATORS** in “Code/Generate the Metrics_2026 Format.R”. The **Step 3** results were reshaped and trimmed to support joining with the square-mile and population denominator tables, retaining only the variables needed for metric generation. Results were written to `"/Data/Results/KEEP LOCAL/From Generate the Metrics/church_2026_form_analysis_ready_08.20.2026.parquet"`.

Table 5: Codebook for New Lookup-Table Fields (Population and Square Miles)

Field	Description
[geoid zcta]_pop	Total population count for each decennial census year, mapped to the corresponding GEOID or ZCTA. GEOID-level values were pre-aggregated to match the census boundary level present in the Step 3 table.
[geoid zcta]_sqMiles	Total square mileage for each decennial census year, mapped to the corresponding GEOID or ZCTA. GEOID-level values were pre-aggregated to match the census boundary level present in the Step 3 table.

Final Results

For the implementation, refer to **SUBSECTION C2: Generate the Metrics in Chunks** in “Code/Generate the Metrics_2026 Format.R”. All **Step 3** results reshaped into the metric calculation form were processed. All chunk results were saved in their respective directory in `"/Data/Results/KEEP LOCAL/From Generate the Metrics/All Metric Results"`.

Table 6: Codebook for All Metric-Algorithm Output Fields

Field	Description
[geoid zcta]	The Census boundary unit to which all metrics in this row have been aggregated: block group, tract, county, or state GEOID, or ZIP Code Tabulation Area (ZCTA).

Continued on next page

Field	Description
religion	The religious classification represented by this row's metrics: e.g. "Christian Churches", "Other Religion", "All Religions".
year	Decennial census reference year: 2000, 2010, or 2020.
[geoid zcta]_pop __YYYY_YYYY	Total resident population for the corresponding decennial census year, assigned to each GEOID or ZCTA. GEOID-level counts were aggregated to match the census boundary granularity.
[geoid zcta]_sqMiles __YYYY_YYYY	Total land area in square miles for the corresponding decennial census year, assigned to each GEOID or ZCTA. GEOID-level values were aggregated to match the census boundary granularity.
n_open__YYYY_YYYY	Total number of ABIs located within the geographic unit and time window. Includes ABIs subsequently excluded by a missingness flag.
closures_[no_moves all] _any__YYYY_YYYY	Count of ABIs with at least one qualifying closure event, defined as four or more consecutive inactive years following at least one active year. no_moves indicates ABIs appearing under multiple addresses within the selected date range were excluded prior to analysis. all indicates all ABIs were retained and compressed: year-activity indicators across all addresses were summed column-wise before event detection.
closures_[no_moves all] _per_sqmi__YYYY_YYYY	Count of ABIs with at least one qualifying closure event, normalised by the total land area (square miles) of the census boundary unit in the corresponding decennial period.
closures_[no_moves all] _per_10k__YYYY_YYYY	Count of ABIs with at least one qualifying closure event, normalised by the total resident population of the census boundary unit in the corresponding decennial period, expressed per 10,000 residents.

Continued on next page

Field	Description
<code>closures_[no_moves all]_avg__YYYY_YYYY</code>	Mean number of closure events recorded by any single ABI within the selected geographic unit and time window.
<code>closures_[no_moves all]_max__YYYY_YYYY</code>	Maximum number of closure events recorded by any single ABI within the selected geographic unit and time window.
<code>reopenings_[no_moves all]_[any avg max]__YYYY_YYYY</code>	Reopening-event counterparts to the closure fields above, sharing identical no_moves/all and any/avg/max semantics. A reopening event is defined as two or more consecutive active years immediately following a qualifying closure.
<code>n_move__YYYY_YYYY</code>	Count of ABIs within the geographic unit that relocated at least once during the selected time window.
<code>moves_total__YYYY_YYYY</code>	Mean number of relocations per ABI within the selected geographic unit and time window.
<code>wavg_dist_km__YYYY_YYYY</code>	Mean across all ABIs of each ABI's move-count-weighted average relocation distance (km). The per-ABI weighted average uses number of moves at each recorded distance as weights; this field reports the mean of those per-ABI values across the geographic unit.
<code>max_dist_km__YYYY_YYYY</code>	Greatest single-move distance (km) recorded among all ABIs within the geographic unit and time window.
<code>move_gt_[5 10 25]mi__YYYY_YYYY</code>	Count of ABIs within the geographic unit that made at least one relocation exceeding the stated distance threshold (5, 10, or 25 miles).
<code>addr1_na__YYYY_YYYY</code>	Missingness flag. <code>address_line_1</code> is NA, preventing address-based geocoding.
<code>addr1_pobox__YYYY_YYYY</code>	Missingness flag. <code>address_line_1</code> contains a PO Box value; the physical location cannot be determined.
<code>[geoid zcta]_na__YYYY_YYYY</code>	Missingness flag. The GEOID or ZCTA value is NA, preventing assignment to a geographic unit.

Continued on next page

Field	Description
NOTE: The suffix YYYY_YYYY represents the start and end years of possible date range textit, where each year is a value between 2000 and 2025 (e.g., 2004_2017).	

Confirm Deliverables Subset

For the implementation, refer to **SUBSECTION D3: Compile Results for Three Date Ranges** in “[Code/Generate the Metrics_2026 Format.R](#)”. Results were compiled for three date range selections: 2000–2006, 2007–2024, and 2010–2015. These were exported as Parquet files across all census boundary levels — block group, tract, county, state, and ZCTA. Additionally, results were further filtered to tract-level GEOIDs corresponding to the city of Milwaukee, WI, exported as a CSV, with the 2010–2015 range visualised as a plot. No new variables from [Final Results](#) were introduced. Results were saved in `./Data/Results/KEEP LOCAL/From Generate the Metrics/PART D Confirm Deliverables`.

References

1. Ushey, K., Wickham, H. & Posit. [Introduction to renv](#).
2. U.S. Census Bureau. TIGER/line shapefiles. Census.gov. <https://www.census.gov/geographies/mapping-files/time-series/geo/tiger-line-file.html> (2025).
3. Pebesma, E. et al. [Units: Measurement units for r vectors](#). (2026).
4. Edzer Pebesma et al. Simple features for r. <https://r-spatial.github.io/sf/> (2018) doi:10.32614/RJ-2018-009.
5. Wickham, H. & RStudio. [Tidyverse: Easily install and load the 'tidyverse'](#). (2023).
6. Kyle Walker & Matt Herman. Tidyensus: Load US census boundary and attribute data as 'tidyverse' and 'sf'-ready data frames. <https://walker-data.com/tidyensus/index.html> (2026).
7. Walker, K. & Rudis, B. [Tigris: Load census TIGER/line shapefiles](#). (2025).